❶ Configures your services in UseLamar() instead of on builder.Services

❷ You can (and should) add ASP.NET Core framework services to the ServiceRegistry, as usual.

❸ Required so that Lamar is used to build your web API controllers

❹ Lamar can automatically scan your assemblies for services to register.

In this example I've used the default conventions to register services. This automatically registers concrete classes and services that are named following expected conventions (for example, `Service` implements `IService`). You can change these conventions or add other registrations in the `UseLamar()` lambda.

The `ServiceRegistry` passed into `UseLamar()` implements `IServiceCollection`, which means you can use all the built-in extension methods, such as `AddControllers()` and `AddAuthorization()`, to add framework services to your container.

WARNING If you're using DI in your Model-View-Controller (MVC) controllers (almost certainly!), and you register those dependencies with Lamar rather than the built-in container, you may need to call `AddControllersAsServices()`, as shown in listing 31.16. This is due to an implementation detail in the way your MVC controllers are created by the framework. For details, see my blog entry titled "Controller activation and dependency injection in ASP.NET Core MVC" at http://mng.bz/aogm.

With this configuration in place, whenever your app needs to create a service, it will request it from the Lamar container, which will create the dependency tree for the class and create an instance. This example doesn't show off the power of Lamar, so be sure to check out the documentation (https://jasperfx.github.io/lamar) and the associated source code for this chapter for more examples. Even in modest-size applications, Lamar can